Differentiate Automatically

VLC = Vision Learning Control

Automatic Differentiation (Part II)

Jonathon Hare

Vision, Learning and Control
University of Southampton

Much of this material is based on this blog post:
`https://rufflewind.com/2016-12-30/reverse-mode-automatic-differentiation`

What is Automatic Differentiation (AD)?

To solve optimisation problems using gradient methods we need to compute the gradients (derivatives) of the objective with respect to the parameters.

- In neural nets we're talking about the gradients of the loss function, $\mathcal{L}$ with respect to the parameters $\boldsymbol{\theta}$: $\nabla_{\boldsymbol{\theta}}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}}$
- AD is important - it's been suggested that "Differentiable programming" could be the term that ultimately replaces deep learning[1].

[1] `http://forums.fast.ai/t/differentiable-programming-is-this-why-we-switched-to-pytorch/9589/5`

What is Automatic Differentiation (AD)?

Computing Derivatives

There are three ways to compute derivatives:

- Symbolically differentiate the function with respect to its parameters
  - by hand
  - using a CAS
- Make estimates using finite differences
- Use Automatic Differentiation

Problems

Static - can't "differentiate algorithms"

Problems

Numerical errors - will compound in deep nets

Reverse Mode AD

- Whilst Forward-mode AD is easy to implement, it comes with a very big disadvantage...
- **For every variable we wish to compute the gradient with respect to, we have to run the *complete program* again.**
- This is obviously going to be a problem if we're talking about the gradients of a function with very many parameters (e.g. a deep network).
- A solution is **Reverse Mode Automatic Differentiation**.

Reversing the Chain Rule

The chain rule is symmetric — this means we can turn the derivatives upside-down:

$$\frac{\partial s}{\partial u} = \sum_i^N \frac{\partial w_i}{\partial u}\frac{\partial s}{\partial w_i} = \frac{\partial w_1}{\partial u}\frac{\partial s}{\partial w_1} + \frac{\partial w_2}{\partial u}\frac{\partial s}{\partial w_2} + \cdots + \frac{\partial w_N}{\partial u}\frac{\partial s}{\partial w_N}$$

In doing so, we have inverted the input-output role of the variables: u is some input variable, the w_i's are the output variables that depend on u. s is the yet-to-be-given variable.

In this form, the chain rule can be applied repeatedly to every input variable u (akin to how in forward mode we repeatedly applied it to every w). Therefore, given some s we expect this form of the rule to give us a program to compute both $\partial s/\partial x$ and $\partial s/\partial y$ in one go...

Reversing the chain rule: Example

$$\frac{\partial s}{\partial u} = \sum_i^N \frac{\partial w_i}{\partial u} \frac{\partial s}{\partial w_i}$$

$x = ?$

$y = ?$

$a = x\,y$

$b = \sin(x)$

$z = a + b$

$$\frac{\partial s}{\partial z} = ?$$

$$\frac{\partial s}{\partial b} = \frac{\partial z}{\partial b}\frac{\partial s}{\partial z} = \frac{\partial s}{\partial z}$$

$$\frac{\partial s}{\partial a} = \frac{\partial z}{\partial a}\frac{\partial s}{\partial z} = \frac{\partial s}{\partial z}$$

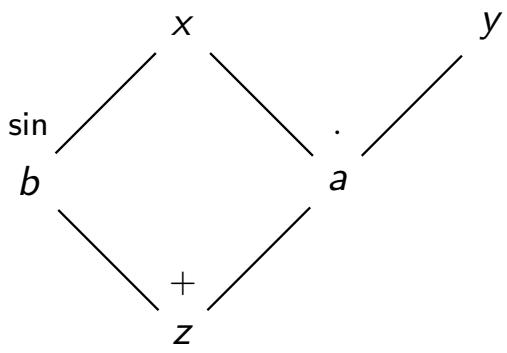
$$\frac{\partial s}{\partial y} = \frac{\partial a}{\partial y}\frac{\partial s}{\partial a} = x\,\frac{\partial s}{\partial a}$$

$$\begin{aligned}\frac{\partial s}{\partial x} &= \frac{\partial a}{\partial x}\frac{\partial s}{\partial a} + \frac{\partial b}{\partial x}\frac{\partial s}{\partial b} \\ &= y\,\frac{\partial s}{\partial a} + \cos(x)\,\frac{\partial s}{\partial b} \\ &= (y + \cos(x))\,\frac{\partial s}{\partial z}\end{aligned}$$

Visualising dependencies

Differentiating in reverse can be quite mind-bending: instead of asking what input variables an output depends on, we have to ask what output variables a given input variable can affect.

We can see this visually by drawing a dependency graph of the expression:



Translating to code

Let's now translate our derivatives into code. As before we replace the derivatives $(\partial s/\partial z, \partial s/\partial b, \dots)$ with variables (`gz`, `gb`, `...`) which we call *adjoint variables*:

```
gz = ?
gb = gz
ga = gz
gy = x * ga
gx = y * ga + cos(x) * gb
```

If we go back to the equations and substitute $s = z$ we would obtain the gradient in the last two equations. In the above program, this is equivalent to setting `gz = 1`.

This means to get the both gradients $\partial z/\partial x$ and $\partial z/\partial y$ we only need to run the program once!

Limitations of Reverse Mode AD

- If we have multiple output variables, we'd have to run the program for each one (with different seeds on the output variables)[2]. For example:

$$\begin{cases} z = 2x + \sin x \\ v = 4x + \cos x \end{cases}$$

- We can't just interleave the derivative calculations (since they all appear to be in reverse)... How can we make this automatic?

[2] there are ways to avoid this limitation...

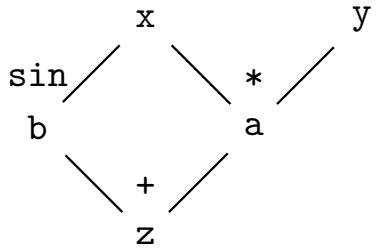
Implementing Reverse Mode AD

There are two ways to implement Reverse AD:

1. We can parse the original program and generate the *adjoint* program that calculates the derivatives.
   - Potentially hard to do.
   - Static, so can only be used to differentiate algorithms that have parameters predefined.
   - But, efficient (lots of opportunities for optimisation)
2. We can make a *dynamic* implementation by constructing a graph that represents the original expression as the program runs.

Constructing an expression graph

The goal is to get something akin to the graph we saw earlier:



The "roots" of the graph are the independent variables `x` and `y`. Constructing these nodes is as simple as creating an object:

```
class Var:
  def __init__(self, value):
    self.value = value
    self.children = []
    ...
  ...

x = Var(0.5)
y = Var(4.2)
```

Each `Var` node can have *children* which are the nodes that depend directly on that node. The children allow nodes to link together in a **Directed Acyclic Graph**.

Building expressions

Each new expression u registers itself as a child of each dependency w_i, together with the local derivative $\partial w_i / \partial u$ used during the reverse sweep:

```
class Var:
  ...
  def __mul__(self, other):
    z = Var(self.value * other.value)

    # weight = dz/dself = other.value
    self.children.append((other.value, z))

    # weight = dz/dother = self.value
    other.children.append((self.value, z))
    return z
  ...
...
# "a" is a new Var that is a child of both x and y
a = x * y
```


Computing gradients

Finally, to get the gradients we need to propagate the derivatives. To avoid unnecessarily traversing the tree multiple times we will *cache* the derivative of a node in an attribute `grad_value`:

```
class Var:
  def __init__(self):
    ...
    self.grad_value = None

  def grad(self):
    if self.grad_value is None:
      # calculate derivative using chain rule
      self.grad_value = sum(weight * var.grad() for weight,
          var in self.children)
    return self.grad_value
        ...
...
a.grad_value = 1.0
print("da/dx = {}".format(x.grad()))
```

Aside: Optimising Reverse Mode AD

- The Reverse AD approach we've outlined is not very space efficient. One way to get around this is to avoid storing the children directly and instead store indices in an auxiliary data structure called a *Wengert list* or *tape*.
- Another interesting approach to memory reduction is trade-off computation for memory of the caches. The Count-Trailing-Zeros (CTZ) approach does just this[3].
- **But,** in reality memory is relatively cheap if managed well...

[3] Andreas Griewank (1992) Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation, Optimization Methods and Software, 1:1, 35-54, DOI: 10.1080/10556789208805505

AD in the PyTorch autograd package

- PyTorch's AD is remarkably similar to the one we've just built:
  - it eschews the use of a tape
  - it builds the computation graph as it runs (recording explicit `Function` objects as the children of `Tensors` rather than grouping everything into `Var` objects)
  - calling `.backward()` runs the reverse sweep and accumulates gradients for leaf tensors in their `.grad` attributes—hence the need to call `zero_grad()` before the next optimisation step.
- PyTorch does some clever memory management to work well in a reference-counted regime and aggressively frees values that are no longer needed.
- The backend is actually mostly written in C++, so its fast, and can be multi-threaded (avoids problems of the GIL).
- It allows easy "turning off" of gradient computations through `requires_grad`.
- In-place operations which invalidate data needed to compute derivatives will cause runtime errors, as will variable aliasing...

Reverse mode computes a VJP

For a function $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$, its Jacobian has shape $m \times n$.
Give the output a seed vector $\boldsymbol{v} \in \mathbb{R}^m$. A reverse sweep computes

$$\underbrace{\bar{\boldsymbol{x}}^\top}_{1\times n} = \underbrace{\boldsymbol{v}^\top}_{1\times m}\underbrace{\boldsymbol{J}_{\boldsymbol{f}}(\boldsymbol{x})}_{m\times n} \quad \Longleftrightarrow \quad \bar{\boldsymbol{x}} = \boldsymbol{J}_{\boldsymbol{f}}(\boldsymbol{x})^\top \boldsymbol{v}.$$

- $\boldsymbol{v}^\top \boldsymbol{J}_{\boldsymbol{f}}(\boldsymbol{x})$ is a **vector–Jacobian product** (VJP).
- The adjoints propagated in our earlier program are the components of $\bar{\boldsymbol{x}}$.
- Reverse mode computes this product without constructing the Jacobian itself.

The seed chooses the output combination

Cotangent vector

A cotangent vector is a vector of sensitivities. Applied to a small output change $\delta\boldsymbol{y}$, it gives the corresponding scalar change $\boldsymbol{v}^\top \delta\boldsymbol{y}$.

The output cotangent $\boldsymbol{v}$ defines a scalar function

$$s(\boldsymbol{x}) = \boldsymbol{v}^\top \boldsymbol{f}(\boldsymbol{x}).$$

Its gradient is exactly the VJP written as a column vector:

$$\nabla_{\boldsymbol{x}} s = \boldsymbol{J}_{\boldsymbol{f}}(\boldsymbol{x})^\top \boldsymbol{v}.$$

- $\boldsymbol{v} = \boldsymbol{e}_i$ selects output f_i; $\boldsymbol{e}_i$ is all zeros except for a 1 in position i.
- An arbitrary $\boldsymbol{v}$ differentiates a weighted sum of the outputs.
- For a scalar output, $m = 1$ and the usual seed is simply $v = 1$.

JVPs and VJPs probe different directions

Forward mode: JVP

$$\boldsymbol{J_f(x)u}$$

- seed an input direction $\boldsymbol{u}$
- one sweep per required Jacobian column

Reverse mode: VJP

$$\boldsymbol{J_f(x)^\top v}$$

- seed an output weighting $\boldsymbol{v}$
- one sweep per required Jacobian row

Deep learning usually has millions of parameters but a scalar loss: reverse mode gives the complete parameter gradient in one sweep.

Computing a VJP with PyTorch

```
import torch
from torch.func import vjp

def f(x):
    return torch.stack((x[0] * x[1], torch.sin(x[0])))

x = torch.tensor([2., 3.])
y, vjp_fn = vjp(f, x)

v = torch.tensor([1., 0.])
(JTv,) = vjp_fn(v)
```

- `y` is $\boldsymbol{f(x)}$.
- `vjp_fn` maps an output cotangent to input cotangents.
- Here `JTv` is $\boldsymbol{J_f(x)^\top v} = [3, 2]^\top$.

https://docs.pytorch.org/docs/stable/generated/torch.func.vjp.html

.backward() performs the same VJP

```
x = torch.tensor([2., 3.], requires_grad=True)
y = f(x)
v = torch.tensor([1., 0.])

y.backward(v)
print(x.grad)       # tensor([3., 2.])
```

- The argument to backward is the output seed $\boldsymbol{v}$.
- For a non-scalar output, it must have the same shape as the output.
- PyTorch performs the reverse sweep and accumulates $\boldsymbol{J_f}(\boldsymbol{x})^\top \boldsymbol{v}$ in x.grad.

Different interface; same mathematics

torch.func.vjp returns the VJP values; .backward() writes them into the gradients of leaf tensors.

A scalar loss supplies the seed for us

```
optimizer.zero_grad()

prediction = model(inputs)
loss = criterion(prediction, targets)

loss.backward()
optimizer.step()
```

- Because loss is scalar, loss.backward() implicitly uses the seed $v = 1$.
- The resulting VJP is the gradient of the loss with respect to every leaf parameter that contributed to it.
- Gradients accumulate in parameter.grad, so they must normally be cleared before the next step.
- The computation graph is freed after the reverse sweep by default.

https://docs.pytorch.org/docs/stable/generated/torch.Tensor.backward.html

Two interfaces, one reverse-mode operation

`torch.func.vjp`

- functional: returns values
- makes the cotangent explicit
- composes with other function transforms
- useful for analysis and higher-order methods

`Tensor.backward()`

- imperative: updates `.grad`
- scalar losses default to seed 1
- gradients accumulate at leaves
- the standard training interface

The common idea

Both propagate an output cotangent backwards through the computation graph to evaluate a VJP.